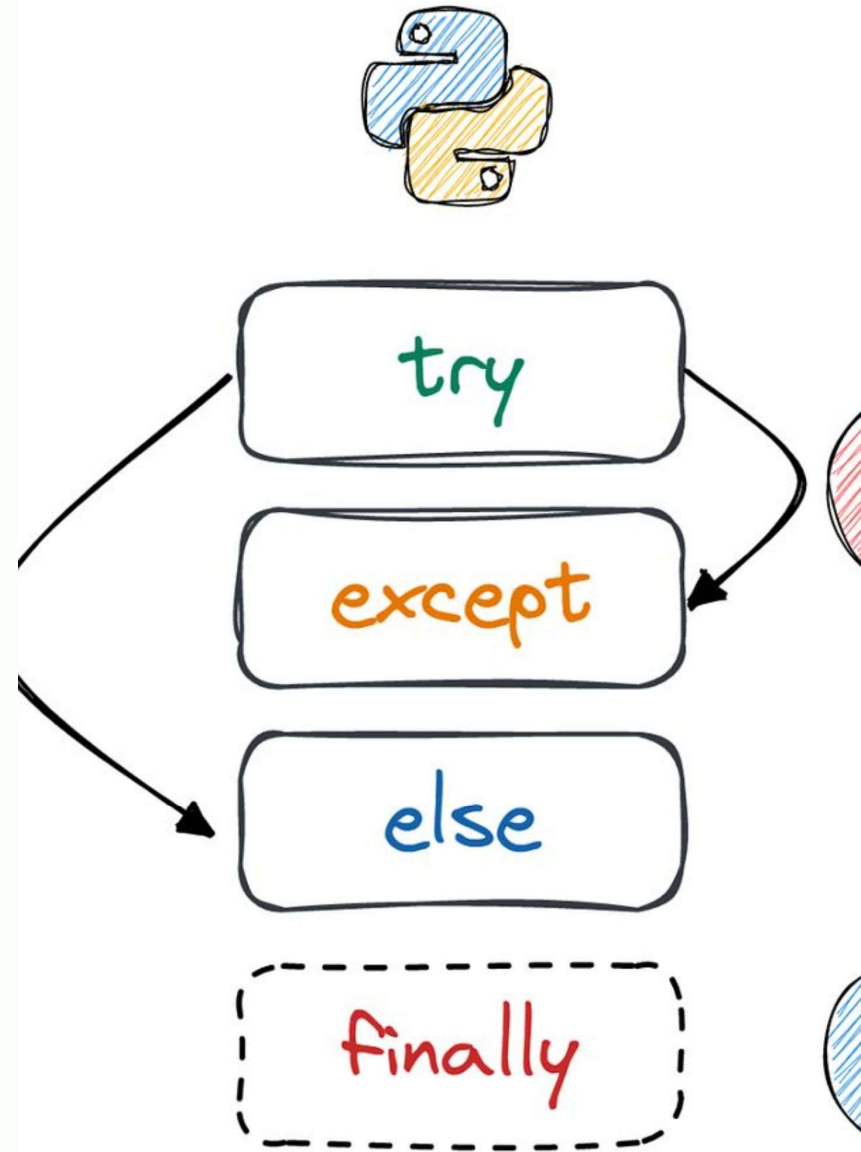


# Errores y Tratamiento de Excepciones en Python

El manejo adecuado de errores es fundamental para crear programas robustos y confiables. En Python, existen dos tipos principales de errores que debemos conocer: los errores de sintaxis, que se detectan antes de la ejecución, y las excepciones, que ocurren durante el tiempo de ejecución del programa.

A lo largo de esta presentación exploraremos en profundidad cómo identificar, manejar y crear nuestros propios mecanismos de control de errores para garantizar que nuestras aplicaciones funcionen de manera estable y predecible, incluso cuando se presentan situaciones inesperadas.



# Errores de Sintaxis: La Primera Barrera

Los errores de sintaxis son expresiones mal formadas que impiden la correcta interpretación del código. A diferencia de los lenguajes compilados como C, que pueden detectar estos errores antes de la ejecución, Python como lenguaje interpretado los descubre línea por línea durante la ejecución.

Cuando el intérprete de Python detecta un error de sintaxis, detiene inmediatamente el proceso y muestra información valiosa para ayudarnos a localizar el problema:

- La ruta del archivo y número de línea donde se detectó el error
- Una reproducción de la línea problemática
- Pequeñas flechas que señalan la ubicación aproximada del error
- Una descripción específica del problema encontrado

Es importante recordar que el lugar donde se detecta el error no siempre coincide con el lugar que debe corregirse, por lo que a veces debemos revisar las instrucciones anteriores.

```
In [1]: print("Hello, World!"  
        Cell In[1], line 1  
          print("Hello, World!"  
                ^  
SyntaxError: incomplete input
```



**Consejo importante:** Si no encuentras el error en la línea indicada, revisa las instrucciones anteriores, especialmente la falta de dos puntos, paréntesis no cerrados o comillas mal emparejadas.

# Excepciones: Errores en Tiempo de Ejecución

Una excepción es un error lógico que se produce durante la ejecución del programa, cuando la sintaxis es correcta pero algo falla en el funcionamiento. Estas son las características principales de las excepciones:

## Interrupción del Flujo

Las excepciones alteran el flujo normal del programa, deteniéndolo en el punto donde ocurre el error a menos que sean manejadas adecuadamente.

## Información Detallada

Proporcionan información específica sobre el tipo de error y su ubicación, incluyendo un traceback que muestra la cadena de llamadas.

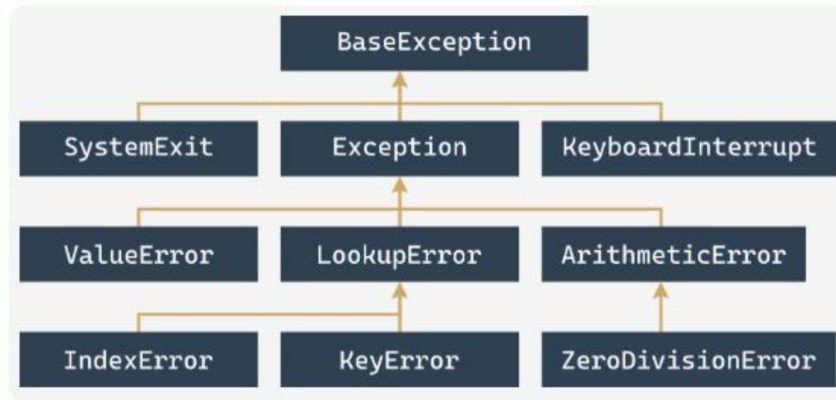
## Manejo Personalizado

Permiten crear excepciones personalizadas y manejar errores de manera específica y controlada según las necesidades del programa.

## Herencia de Clases

Las excepciones en Python son clases que forman una jerarquía, permitiendo crear sistemas organizados de manejo de errores.

Las excepciones se utilizan para señalar condiciones excepcionales como división por cero, referencias a variables no definidas, o apertura de archivos inexistentes. Python 3 define 63 excepciones integradas organizadas en una jerarquía en forma de árbol.



Por ejemplo, `ZeroDivisionError` es un caso especial de `ArithmeticError`, que a su vez es un caso especial de `Exception`. Esta organización permite capturar errores aritméticos en general o dividir por cero específicamente.

Algunos ejemplos de excepciones son los siguientes:

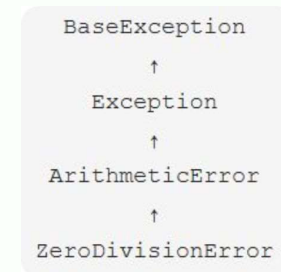
```
ZeroDivisionError: division by zero
NameError: name 'zzzzzz' is not defined
TypeError: can only concatenate str (not "int") to str
```

Esto es una pequeña sección del árbol completo de excepciones. Comencemos examinando el árbol desde la hoja `ZeroDivisionError`.

— `ZeroDivisionError` es un caso especial de una clase de excepción más general llamada `ArithmeticError`.

— `ArithmeticError` es un caso especial de una clase de excepción más general llamada solo `Exception`.

— `Exception` es un caso especial de una clase más general llamada `BaseException`.



Veamos un ejemplo, aunque posteriormente explicaremos en detalle los distintos tipos de excepciones:

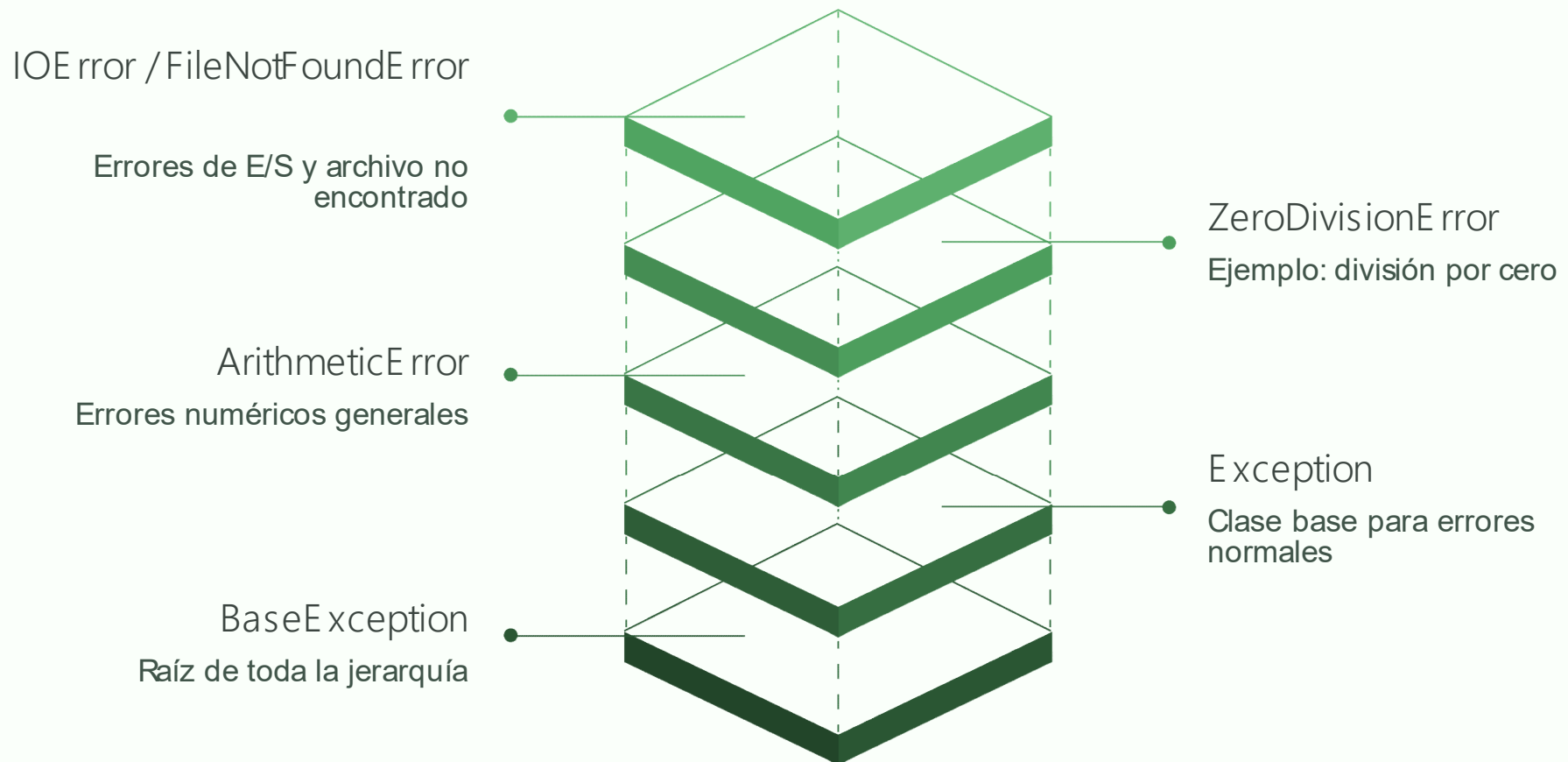
```
'6' + 17
Traceback (most recent call last):
  File "main.py",line 1, in <module>
    '6' + 17
TypeError: can only concatenate str (not "int") to str
```

En este caso intentamos concatenar una cadena '6' con un valor entero 17. Aunque sintácticamente está bien construida la instrucción, el intérprete de Python nos dará un error, indicativo de que la línea 1 tiene un error de tipo, representado por `TypeError`, y concretamente indicará que solo puede concatenar cadenas, no una cadena y un entero. Para poder concatenar una cadena y un entero es necesario hacer un casting sobre el entero para convertirlo previamente en cadena.

```
# Ejemplo de TypeError
mensaje = '6' + 17 # Error: no se puede concatenar str y int
# Solución
mensaje = '6' + str(17) # Correcto: ambos son strings
```

# Jerarquía de Excepciones en Python

Se ha visto que Python organiza sus excepciones en una estructura jerárquica que permite un manejo más eficiente y específico de los errores. Esta jerarquía funciona como un árbol invertido, donde las excepciones más generales están en la raíz y las más específicas en las hojas.



## Excepciones Base

- `BaseException`: Clase raíz de todas las excepciones, incluidas las de control del sistema
- `Exception`: Clase base para excepciones que no indican salida del intérprete

## Ventajas de la Jerarquía

- Permite capturar grupos de excepciones relacionadas
- Facilita el manejo genérico o específico según necesidades
- Proporciona flexibilidad en el diseño de sistemas de error

# Tipos de Excepciones Estándar

Python incluye una amplia gama de excepciones estándar que cubren los errores más comunes en programación. Conocer estas excepciones es esencial para escribir código robusto.



## Excepciones Aritméticas

- `ArithmeticError`: Clase base para errores de cálculo numérico. Es la clase base para las excepciones `FloatingPointError`, `OverflowError` y `ZeroDivisionError`.
- `ZeroDivisionError`: División o módulo por cero
- `OverflowError`: Resultado aritmético demasiado grande
- `FloatingPointError`: Error en cálculo de punto flotante

```
result = 21/0
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    result = 21/0
ZeroDivisionError: division by zero
```

Podríamos haber controlado este error de la siguiente forma:

```
try:
    resultado = 21/0
except ZeroDivisionError as e:
    # Salida utilizando F-strings
    print(f"Error de división por cero: {e}")
```



## Excepciones de E/S

- **IOError**: Clase base para errores de entrada/salida
- **FileNotFoundError**: Archivo o directorio no encontrado
- **PermissionError**: Operación sin permisos adecuados

```
f = open("fichero.txt", "r")
Traceback (most recent call last):
File "main.py", line 1, in <module>
f = open("fichero.txt", "r")
FileNotFoundError: [Errno 2] No such file or directory: 'fichero.txt'
```

Podríamos haber controlado este error de la siguiente forma:

```
try:
    with open('archivo_que_no_existe.txt', 'r') as archivo:
        contenido = archivo.read()
except FileNotFoundError as e:
    print(f"Archivo no encontrado: {e}")
```



## Excepciones de Argumentos

- **TypeError:** Operación aplicada a tipo inapropiado
- **ValueError:** Tipo correcto pero valor inapropiado

```
4 + "c"
```

```
Traceback (most recent call last):
```

```
File "main.py", line 1, in <module>
```

```
4 + "c"
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Podríamos haber controlado este error de la siguiente forma:

```
try:
```

```
    numero = int("abc")
```

```
except ValueError as e:
```

```
    print(f"Error de valor: {e}")
```



## Excepciones de Variables y Atributos

- **NameError:** Variable local/global no encontrada
- **AttributeError:** Atributo inexistente en objeto

```
cadena = "Programación en Python"
cadena.get(i)
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    cadena.get(i)
AttributeError: 'str' object has no attribute 'get'
```

```
try:
    print(variable_no_definida)
except NameError as e:
    print(f"Nombre de variable no encontrado: {e}")
```



## Excepciones del sistema

- **SystemError**: Se lanza cuando se detecta un error interno en Python
- **KeyboardInterrupt**: Se lanza cuando se interrumpe la ejecución del programa mediante la entrada del usuario (por ejemplo, Ctrl+C)

```
while True: print("Python")
Python
Python
Python
PythonTraceback (most recent call last):
  File "main.py", line 1, in <module>
    while True: print("Python")
    ...
KeyboardInterrupt
```

# Gestión Básica con try-except

La gestión de excepciones en Python se realiza mediante la estructura **try-except**, que permite interceptar errores y manejarlos de manera controlada sin detener abruptamente el programa.

## Estructura Básica

```
try:
    # Código que puede fallar
    resultado = 10 / 0
except ZeroDivisionError:
    # Manejo específico del error
    print("Error: División por cero")
except Exception as e:
    # Manejo genérico
    print(f"Error inesperado: {e}")
```

## Mejores Prácticas

- Ser específico con las excepciones capturadas
- Usar múltiples bloques except para diferentes errores
- Incluir un except genérico al final como red de seguridad
- Evitar usar except sin especificar el tipo de excepción

③ **Consejo:** Es recomendable gestionar las excepciones de manera específica escribiendo varias cláusulas except para el mismo try, permitiendo un manejo más preciso de cada tipo de error.

01

### Ejecutar código en try

Python ejecuta las instrucciones del bloque try normalmente

02

### Detectar excepción

Si ocurre un error, busca el bloque except correspondiente

03

### Manejar el error

Ejecuta el código del bloque except y continúa el programa

# Bloques else y finally: Control Completo

Los bloques `else` y `finally` proporcionan control adicional sobre el flujo de ejecución en el manejo de excepciones, permitiendo ejecutar código específico según si ocurrieron errores o no.



## Bloque try

Se ejecuta el código que puede generar excepciones



## Bloque except

Se ejecuta solo si ocurre una excepción específica



## Bloque else

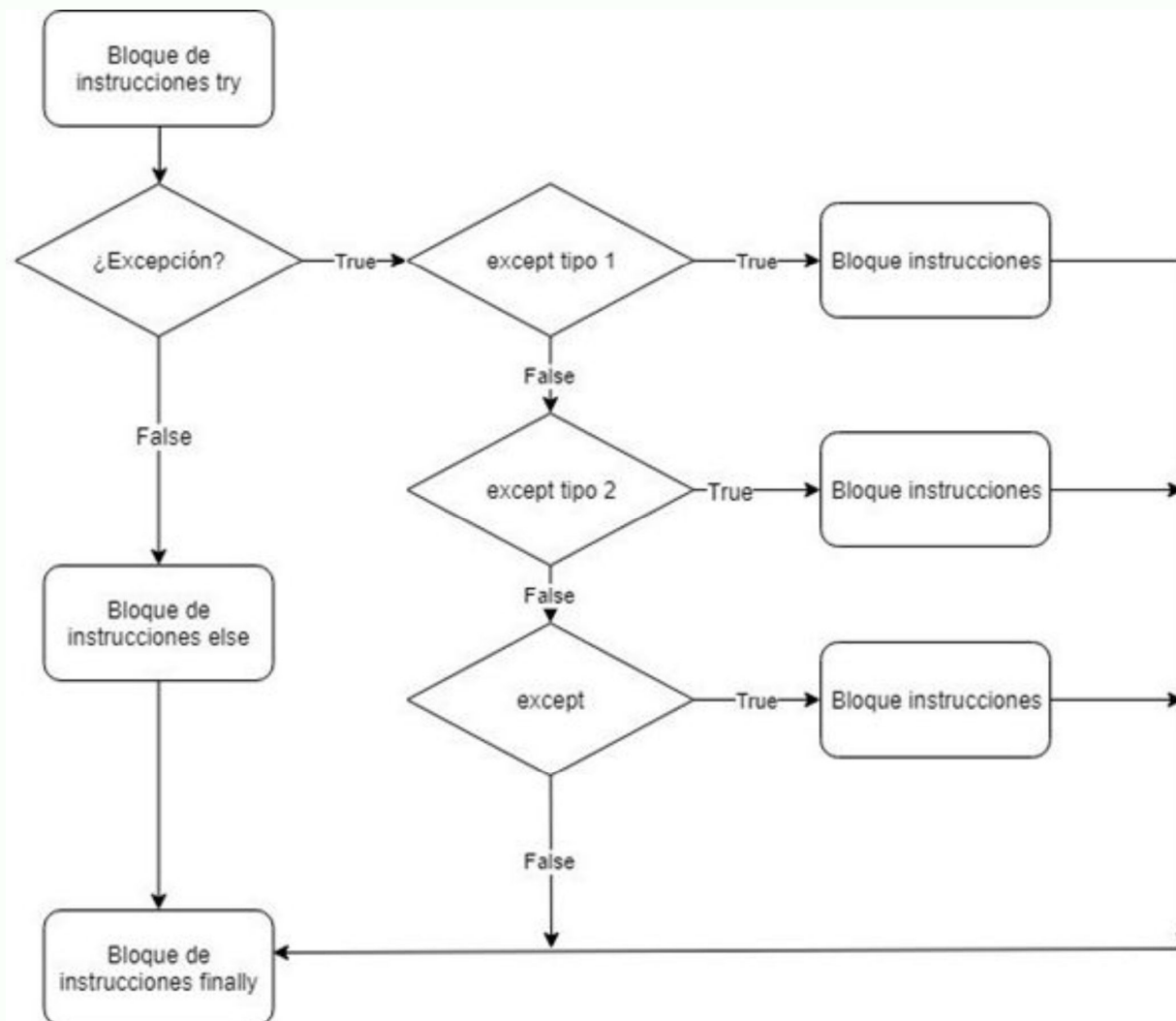
Se ejecuta solo si NO ocurre ninguna excepción



## Bloque finally

Se ejecuta SIEMPRE, haya o no excepciones

```
try:
    [instrucciones]
except <tipo de la excepción>:
    [instrucciones si ocurre esa excepción]
else:
    [instrucciones si no ocurre ninguna excepción]
finally:
    [instrucciones ocurran o no excepciones]
```



Este ejemplo muestra cómo el bloque **finally** garantiza que los recursos se liberen correctamente, independientemente de si ocurrieron errores, mientras que **else** permite ejecutar código adicional solo cuando todo funciona correctamente.

```
try:
    archivo = open("resultado.txt", "w")
    # Suponemos que el archivo existe
    print("Archivo resultado.txt abierto.")
    resultado = 15 * (3/0)
except IOError:
    # Instrucciones si ocurre la excepción IOError
    print("Error de entrada/salida.")
except ZeroDivisionError:
    # Instrucciones si ocurre la excepción
    ZeroDivisionError
    print("Error división por cero.")
else:
    # Instrucciones si no ocurre ninguna excepción
    print("El resultado de la división es", resultado)
    archivo.write(resultado)
finally:
    # Instrucciones si ocurren o no ocurren excepciones
    if not(archivo.closed): archivo.close()
    print("Archivo resultado.txt cerrado.")
```

Si ejecutamos el código del programa, obtendremos el siguiente resultado:

```
Archivo resultado.txt abierto.  
Error división por cero.  
Archivo resultado.txt cerrado.
```

Como puedes observar, ha tenido lugar un error debido a una división por cero, pero este ha sido controlado, informando del error producido. Además, gracias al bloque finally, se ha podido cerrar el fichero, a pesar de la excepción lanzada. No se ejecutan las instrucciones del bloque else, pero sí las del bloque finally.

# Lanzando Excepciones con raise

La palabra clave **raise** se utiliza para lanzar (o propagar) una excepción en Python. Al usar raise, podemos generar errores personalizados o volver a lanzar excepciones capturadas para que sean manejadas en un nivel superior del programa.

La instrucción raise en Python se utiliza para lanzar una excepción intencionalmente. Esto puede ser útil en varias situaciones, como:

- **Indicar errores en el flujo de programa:** Cuando se detecta una condición que no puede ser manejada adecuadamente en el contexto actual.
- **Forzar el manejo de errores:** Asegurar que los errores sean tratados adecuadamente en la capa superior de la aplicación.

## Sintaxis Básica

```
raise ExceptionType("Mensaje de error")
```

- **ExceptionType** es la clase de excepción que deseas lanzar.
- **Mensaje de error** es una cadena opcional que proporciona detalles sobre el error.

## Ejemplo de uso básico

```
def verificar_edad(edad):  
    if not isinstance(edad, (int, float)):  
        raise TypeError("La edad debe ser un número")  
    if edad < 0:  
        raise ValueError("La edad no puede ser negativa")  
    if edad > 150:  
        raise ValueError("Edad no válida: demasiado alta")  
    return True  
  
# Uso con manejo de excepciones  
try:  
    verificar_edad(-5)  
except ValueError as e:  
    print(f"Error de validación: {e}")  
except TypeError as e:  
    print(f"Error de tipo: {e}")
```

# Excepciones Personalizadas

En Python, además de manejar las excepciones integradas, también podemos definir nuestras propias excepciones personalizadas. Esto es útil porque podemos representar errores específicos en la lógica de nuestra aplicación, lo que nos permite gestionar estos errores con mayor precisión.



## Herencia de Exception

Las excepciones personalizadas deben heredar de la clase Exception o alguna de sus subclases



## Constructor Personalizado

Agregar un constructor que acepte mensajes de error y parámetros adicionales relevantes



## Jerarquía de Errores

Crear una jerarquía propia de excepciones con herencia entre ellas para organizar mejor los errores

## Definición Básica

Para crear una excepción personalizada, simplemente definimos una nueva clase que hereda de Exception.

```
class MiExcepcionPersonalizada(Exception):  
    def __init__(self, mensaje):  
        super().__init__(mensaje)
```

- ✔ Ventaja: Las excepciones personalizadas hacen que el código sea más legible y permiten un manejo más específico de errores relacionados con la lógica de negocio.

Es común agregar un constructor que acepte un mensaje de error y otros posibles parámetros relevantes. También podemos añadir cualquier número de parámetros adicionales que necesitemos:

```
class ErrorDeValidacion(Exception):  
    def __init__(self, campo, mensaje):  
        self.campo = campo  
        self.mensaje = mensaje  
        super().__init__(f"Error en '{campo}': {mensaje}")  
  
# Lanzar la excepción  
raise ErrorDeValidacion("nombre", "El nombre no puede estar vacío.")
```

En el siguiente ejemplo, solicitamos al usuario que introduzca el carácter c para continuar o el carácter f para finalizar. Sin embargo, no podemos asegurar que siempre vaya a introducir alguno de estos caracteres. Por ello, creamos nuestra propia excepción (líneas 1-6), a la que llamamos MiExcepcion. De esta manera, si el usuario introduce un carácter distinto a c y f se lanzará la excepción creada y se mostrará un mensaje de error indicando que el carácter introducido no es válido.

La gestión de excepciones es el mecanismo principal para garantizar programas «robustos», término utilizado para las soluciones capaces de hacer frente a diversas contingencias. Un programa robusto hará una gestión adecuada de los potenciales casos de error para evitar la brusca interrupción de su ejecución.

```
class MiExcepcion(Exception):
    def __init__(self, valor):
        self.valor = valor

    def __str__(self):
        return "Error: " + str(self.valor)

try:
    fin = False
    while not fin:
        entrada = input("Introduzca c para continuar o f
        para finalizar:")
        if entrada != "f" and entrada != "c":
            raise MiExcepcion(entrada + " no es un valor
            válido.")
        elif entrada == "f":
            fin = True
except MiExcepcion as e:
    print (e)
```

# Jerarquía de Excepciones Personalizadas

Una jerarquía de excepciones personalizadas en Python es un conjunto de clases de excepciones definidas por el usuario que heredan de otras excepciones para crear una estructura organizada, permitiendo capturar errores específicos o grupos de errores. Un ejemplo práctico incluye una clase base `ErrorDeApp` y subclases como `ErrorDeConexion` y `ErrorDeDatos`, cada una heredando de `Exception` y proporcionando mensajes personalizados al ser lanzadas.

## 1 Definir la clase base

Se crea una excepción general para el dominio de tu aplicación.

```
class ErrorDeApp(Exception):  
    """Excepción base para todos los errores de la aplicación."""  
    pass
```

## 2 Definir subclases

Se crean excepciones más específicas que heredan de la clase base o de otras subclases.

```
class ErrorDeConexion(ErrorDeApp):
    """Error al intentar establecer una conexión."""
    def __init__(self, mensaje="No se pudo establecer la conexión"):
        self.mensaje = mensaje
        super().__init__(self.mensaje)

class ErrorDeDatos(ErrorDeApp):
    """Error relacionado con los datos de la aplicación."""
    pass

class ErrorDeFormato(ErrorDeDatos):
    """Error de formato incorrecto en los datos."""
    def __init__(self, mensaje="Formato de datos incorrecto"):
        self.mensaje = mensaje
        super().__init__(self.mensaje)
```

### 3 Lanzar y capturar las excepciones

```
def conectar_a_servicio(url):
    if "mal" in url:
        raise ErrorDeConexion("URL
        malformada")
    print(f"Conectando a {url}...")
    # Simular un error de conexión
    if "servicio_caído" in url:
        raise ErrorDeConexion("El servicio está caído")

def procesar_datos(data):
    if not isinstance(data, dict):
        raise ErrorDeFormato("Los datos deben ser un diccionario")
    print("Procesando datos...")
    # Simular otro error de datos
    if "error" in data:
        raise ErrorDeDatos("Error interno en el procesamiento de datos")

# --- Bloque try-except ---
try:
    conectar_a_servicio("http://servicio_caído")
    procesar_datos({"nombre": "Ejemplo"})
except ErrorDeConexion as e:
    print(f"Error de conexión: {e}") # Capta
    ErrorDeConexion
except ErrorDeFormato as e:
    print(f"Error de formato: {e}") # Capta
    ErrorDeFormato
except ErrorDeDatos as e:
    print(f"Error de datos: {e}") # Capta
    ErrorDeDatos
except ErrorDeApp as e:
    print(f"Un error genérico de la aplicación
    ocurrió: {e}") # Capta otros ErroresDeApp
```

# Ejemplos Prácticos y Mejores Prácticas

La aplicación efectiva del manejo de excepciones requiere conocer patrones comunes y seguir buenas prácticas que garanticen programas robustos y mantenibles. Estos ejemplos demuestran implementaciones reales y profesionales.



Ocurrió un error al generar esta imagen

## Validación de E ntrada

```
class EntradaInvalidaError(Exception):  
    pass  
  
def procesar_edad(edad):  
    if edad < 0:  
        raise EntradaInvalidaError("La  
edad no puede ser negativa")  
    print(f"Edad válida: {edad}")
```



Ocurrió un error al generar esta imagen

## Autenticación

```
class AutenticacionError(Exception):  
    pass  
  
def autenticar(usuario, clave):  
    if usuario != "admin" or clave !=  
    "1234":  
        raise AutenticacionError(  
            "Credenciales incorrectas")
```



Ocurrió un error al generar esta imagen

## Recursos No E ncontrados

```
class RecursoNoEncontrado(Exception):  
    pass  
  
def obtener_recurso(id_recurso):  
    recursos = {1: "A", 2: "B"}  
    if id_recurso not in recursos:  
        raise RecursoNoEncontrado(f"ID  
{id_recurso} no existe")  
    return recursos[id_recurso]
```

## Principios de Programas Robustos

### Especificidad en el Manejo

Capturar excepciones específicas en lugar de usar except genéricos. Esto permite respuestas apropiadas para cada tipo de error.

### Logging y Monitoreo

Registrar excepciones con información contextual suficiente para facilitar el diagnóstico y mantenimiento del sistema.

### Liberación de Recursos

Usar siempre bloques finally para garantizar que recursos como archivos, conexiones de base de datos y sockets se cierren correctamente.

### Graceful Degradation

Implementar mecanismos de recuperación que permitan al programa continuar funcionando parcialmente cuando sea posible.

📌 **Recordatorio:** La gestión de excepciones es el mecanismo principal para garantizar programas robustos, capaces de hacer frente a diversas contingencias sin interrumpir bruscamente su ejecución.